PRODUCT ANALYSIS · UI/UX VISION · AI SECURITY ROADMAP

WASA

Web Application Security Automation

Thesis prototype → Enterprise-ready AI security platform

Current-state analysis · Experience redesign · Capability expansion

Designed for founders, security engineers, and product teams

Prepared as a product vision document for WASA

Stack today: React · Flask · Wappalyzer · Vulners · Nmap · Paramiko

Think bigger than the code. Build the security OS teams actually use.

01 · Executive Summary

What WASA is today

WASA (Web Application Security Automation) is a master's thesis full-stack product that lets a user paste a target URL and run four security checks from a browser UI: **Reflected XSS**, **Error-based SQL Injection**, **Web technology fingerprinting with CVE lookup**, and **network-level credential attack on SSH/Telnet** (with post-exploitation file transfer intent). The experience is intentionally simple — one URL field and four action buttons — but under the hood it already combines frontend orchestration (React + MUI), a Flask API, and Python security tooling (BeautifulSoup, Wappalyzer, Vulners, nmap, paramiko).

The strategic opportunity is large: the scaffolding of a **security automation product** exists, but the UX, scanner depth, safety controls, data model, and AI layer are still at prototype stage. This document treats WASA not only as current code, but as a product that can become a modern **AI-assisted AppSec + light Attack Surface Management** platform for students, freelancers, MSSPs, and engineering teams who need fast, explainable vulnerability feedback.

Current value	Primary gaps
One-click multi-module scans	UI is functional, not product-grade
Web + network coverage mix	Shallow payloads / false positives
CVE intel via Vulners	No auth, jobs, history, reports
Thesis-grade end-to-end demo	Dangerous network abuse surface

North-star product statement

“WASA is the fastest path from a URL to a trusted, explainable security brief — with AI that triages noise, narrates attack paths, and drafts fixes developers can ship.”

Figure 1 — Brand / product mood: protection, automation, modern SaaS.

02 · Current System Analysis

Architecture as implemented

Today's WASA is a **two-process system** with Create React App on the frontend (proxying to Flask on port 5000). The React app (`src/App.js`) owns all scan UX state. Flask routes dispatch to Python modules for each scan type. An unused Express helper and an alternate `SecurityTester` component indicate exploratory iterations.

Layer	Responsibility	Primary file
React UI	URL input + 4 scan buttons + result renderers	src/App.js
Flask API	POST /scanXss, /scanSQLI, /detectTechnologies, /scanNetwork	api/venv/api.py
XSS engine	Find forms, inject <script>alert, check reflection	xssTest.py
SQLi engine	Append ' / " ; detect SQL error strings	sqliTest.py
Tech intel	Wappalyzer versions → Vulners softwareVulnerabilities	api.py helpers
Network	Resolve host → nmap 22/23 → paramiko/telnet brute → optional SFTP put	newNetAttack.py

Feature-by-feature capability audit

Module	Maturity	Assessment
Reflected XSS	Medium	Forms only; single payload; no DOM/stored/context awareness
SQL Injection	Low–Med	Error-based only; tiny payload set; no blind/time-based/UNION
Web Tech + CVE	Medium	Solid idea; depends on version detection quality + API key
Network / Post-ex	High risk	Credential stuffing on 22/23 + file drop — powerful but unsafe without guardrails
Reporting	None	Results are ephemeral UI text only
Auth / multi-user	None	Open API; CORS * ; no tenancy
Job system	None	Synchronous long scans can hang the browser

Critical engineering & safety findings

- **Hardcoded third-party API key** in source (Vulners) — rotate immediately; move to secrets manager / env.
- **Committed virtualenv** under `api/venv` bloats the repo and ships binaries/packages — replace with requirements.txt + clean install.
- **Network module performs real credential attacks** and can transfer files — must be gated by ownership proof, allowlists, and explicit lab mode.
- **No authentication / authorization** turns the product into an open scanning proxy if exposed publicly.
- **SQLi detector imports re inconsistently** (uses re without import in active path) — reliability risk.
- **UX state bugs** (e.g. web-tech empty result still marks testing true; mixed MUI v4/v5 styling) reduce trust.
- **Results are not structured** (free-text / ad-hoc objects) — blocks history, triage, and AI later.

User experience of the current UI

The current interface is a **developer demo console**, not a security product. There is no brand shell, no progressive disclosure, no severity language, no empty/loading/error system design, and no path from finding → evidence → remediation. Four green buttons compete for attention with equal weight even though their risk profiles are radically different (XSS vs live network brute force). Visual design relies on light-blue body background and

default Material inputs — functional, but not memorable or trustworthy for security buyers.

03 · UI/UX Design Vision

Design principles

Clarity over cleverness. Every scan type states what it does, what it touches, and residual risk before launch.

Severity-first hierarchy. Critical findings and live exploitability always surface above raw logs.

Evidence you can defend. Each finding has request/response proof, confidence, and CWE/OWASP mapping.

Safe by default. Network/post-ex modules require explicit lab mode + target ownership confirmation.

AI as copilot, not black box. Models explain ranking and propose fixes; humans stay in control.

Calm dark operations aesthetic. Navy/cyan system UI signals precision; reduce visual noise during long scans.

Proposed information architecture

- **Dashboard** — risk score, recent scans, trend sparklines, module health
- **New Scan wizard** — target → scope → modules → safety checks → launch
- **Findings inbox** — filterable by severity, asset, status, assignee
- **Finding detail** — evidence, AI narrative, remediations, retest
- **Attack surface map** — hosts, ports, tech stack, trust boundaries
- **AI Assistant** — chat over a scan corpus; “what should I fix first?”
- **Reports** — executive PDF, developer markdown, compliance exports
- **Integrations & Settings** — Jira/Slack/SIEM, API keys, org policy

Figure 2 — Visual direction: dark ops, cyan/indigo accents, modular cards.

Command Center UI (proposed)

The redesigned home is a **Security Command Center**: sticky scan bar, KPI strip (Critical / High / Medium / Risk Score), module grid with enable/disable + roadmap tags, and a priority findings rail auto-ranked by severity × confidence × exploitability. This replaces the flat four-button layout with progressive structure while preserving one-URL speed.

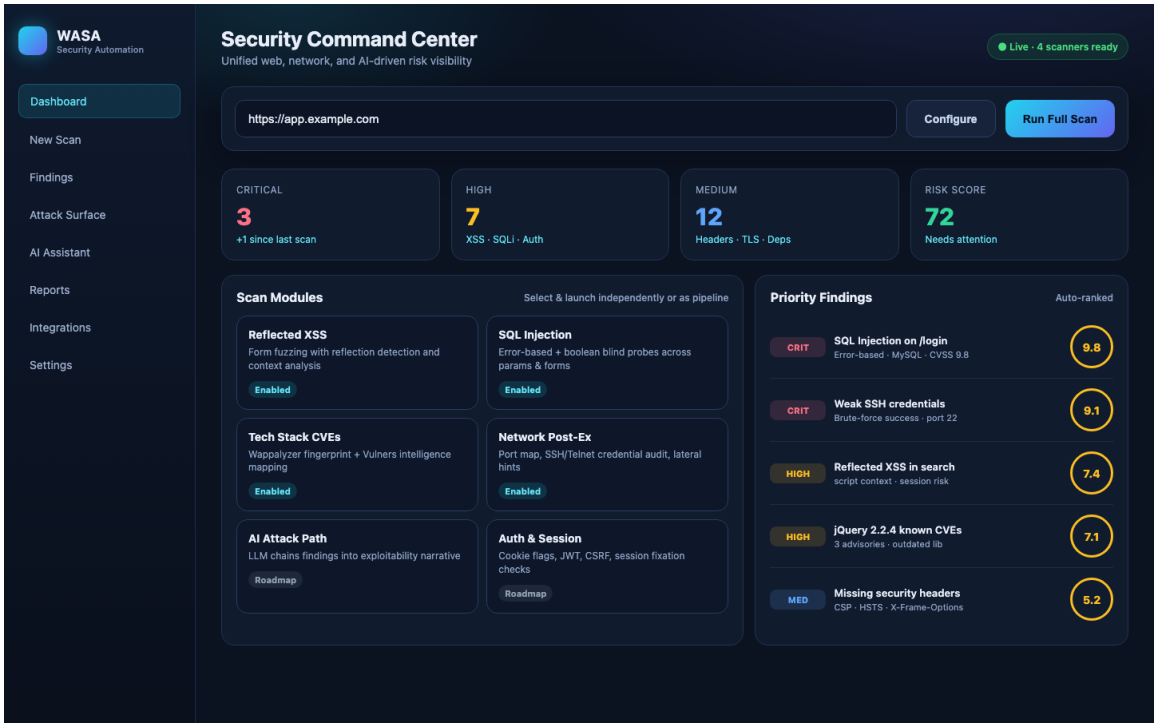

Figure 3 — High-fidelity dashboard mock: modules, KPIs, and prioritized findings.

Finding detail + AI remediation workspace

Security products win when a finding page answers four questions in under 30 seconds: **What is wrong? How bad is it? Prove it. How do I fix it?** The proposed detail view pairs classic evidence (HTTP request/response, parameters, CWE/OWASP) with an **AI Security Copilot** panel that writes an attack narrative, business impact, and copy-paste fix snippets in the project's language.

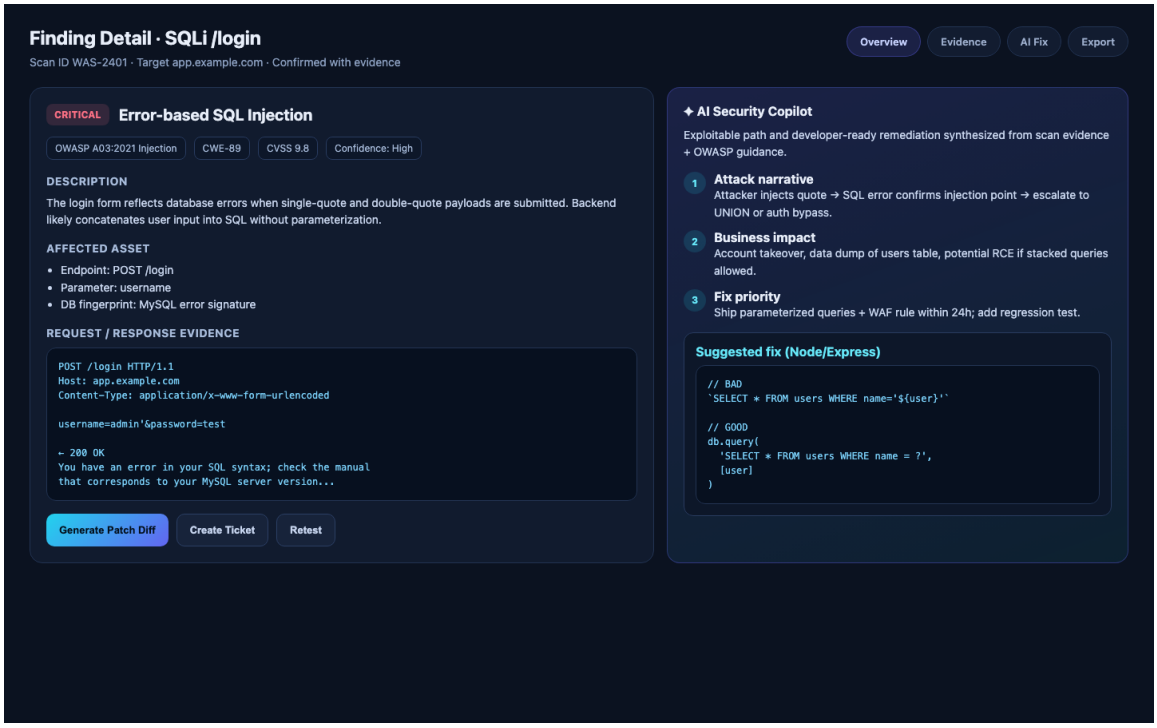

Figure 4 — Finding detail with evidence and AI copilot remediation.

Micro-UX specifications

Moment	Specification
Empty state	Illustrate first scan; sample demo target; estimated time per module
Loading	Per-module progress stream; cancel; partial results as they arrive
Errors	Human messages + retry; never dump stack traces to end users
Safety modal	For network scans: confirm IP ownership, lab-only checkbox, rate limits
Severity chips	CRIT / HIGH / MED / LOW with consistent color tokens
Accessibility	WCAG AA contrast, keyboard scan launch, focus rings, reduced motion
Mobile	Read findings on phone; launch full scans on desktop/tablet primarily

04 · Functionality Expansion (Beyond Current Code)

Think like an AI cybersecurity product leader

WASA should evolve from four independent scripts into a **modular scanner fabric** with shared crawling, authentication contexts, evidence storage, and policy. Below is a capability backlog that intentionally goes far beyond the thesis implementation — because the market gap is not “another XSS button,” it is trustworthy automation + explanation + developer action.

A. Deepen existing modules

- **XSS:** multi-context payloads (HTML, attribute, JS, URL); DOM XSS via headless browser; stored XSS crawl; CSP bypass checks; polyglots; mutation XSS.
- **SQLi:** boolean & time-based blind; UNION column enumeration; second-order; JSON/API parameter injection; ORM-aware heuristics; out-of-band where allowed.
- **Tech/CVE:** CPE normalization; EPSS + KEV prioritization; SBOM for detected JS libs; exploit-available flags; patch version guidance.
- **Network:** full service fingerprinting (not only 22/23); safe banner grab; credential checks only in authenticated lab mode; remove automatic file drop from default path.

B. New AppSec scanner packs

Pack	Example checks
Auth & session	Cookie flags, JWT alg confusion, session fixation, logout invalidate, password reset poisoning
CSRF / CORS	Token presence, SameSite, preflight reflection, null origin issues
SSRF	Webhook/URL fetch parameters, cloud metadata probe (lab-only)
Access control	IDOR heuristics, forced browsing, role swap testing with dual sessions
Injection+	Command, template (SSTI), LDAP, header injection, CRLF
Client secrets	API keys in JS bundles, source maps, exposed .git/.env
API security	OpenAPI-driven fuzzing, mass assignment, rate-limit absence, GraphQL introspection abuse
TLS & headers	HSTS, CSP quality score, cookie security, mixed content
Upload & files	Content-type bypass, path traversal, unrestricted extension
Business logic	Race conditions, coupon/price tamper patterns (semi-auto with AI hints)

C. Attack Surface Management (light ASM)

- Domain → subdomain discovery, live host detection, screenshotting, tech inventory over time.
- Change detection: new port, new tech version, new endpoint → auto re-scan.
- Asset ownership tags and environment labels (prod / staging / lab).

D. Platform product features

- Async job queue, scan history, multi-target projects, RBAC, SSO.
- Evidence vault (requests, HTML snapshots, HAR) with retention policies.
- Exports: executive PDF, SARIF for GitHub code scanning, CSV, JSON API.
- Integrations: Jira, Linear, Slack, Teams, SIEM webhooks, GitHub PR comments.
- Policy engine: fail CI if Critical > 0; allowlist false positives; compliance maps (OWASP Top 10, PCI themes).

05 · AI Cybersecurity Layer

Where AI multiplies WASA

Scanners produce volume; humans need judgment. AI should sit **above** deterministic engines — never replace evidence-based detection as the sole truth. The winning pattern is: **classic engines find → AI explains, prioritizes, and remediates drafts → human approves.**

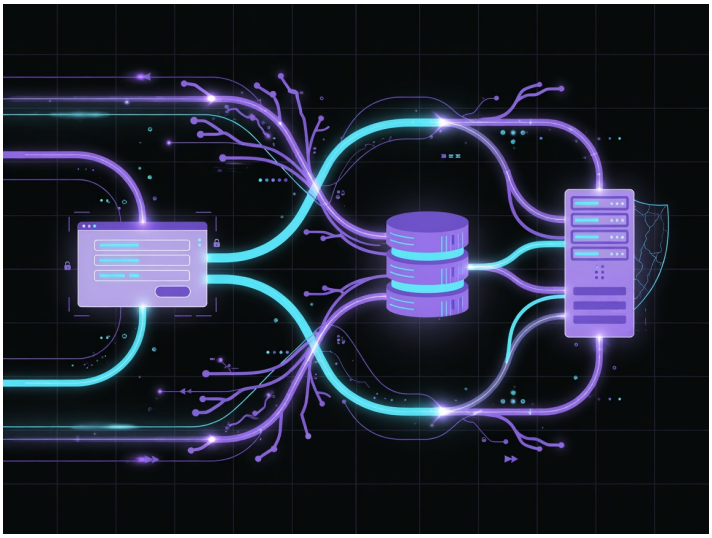

Figure 5 — AI correlating assets into attack-path reasoning.

AI feature set

AI capability	Product behavior
Smart triage	Cluster duplicates, suppress known FP patterns, confidence calibration
Attack path graph	Chain XSS → session theft → admin; or SQLi → data access narratives
Risk scoring	Blend CVSS, EPSS, asset criticality, exposure, and exploit chatter
Natural language brief	CISO summary + developer ticket body in one click
Fix generation	Language-aware patches, unit tests, WAF virtual patch rules
Payload synthesis	LLM proposes next payloads when baseline checks fail (sandbox)
Chat with scan	RAG over findings + evidence: “Any path to PII?”
Threat intel fusion	Map tech stack to active campaigns / CISA KEV
Anomaly reviews	Diff consecutive scans; highlight meaningful deltas only
Compliance mapping	Auto tag controls (OWASP ASVS themes, ISO-style statements)

Responsible AI & safety constraints

- Never auto-execute destructive post-exploitation without multi-step confirmation and lab flag.
- Show model uncertainty; separate **observed evidence** from **inferred narrative**.
- Redact secrets in logs/prompts; do not train on customer traffic without contract.
- Rate-limit AI-generated active testing; default to passive analysis on production scopes.
- Maintain audit trail: who ran what, against which target, with which modules.

Figure 6 — Future SOC-style visualization for multi-target risk.

06 · Target Architecture

From prototype to platform

The target architecture separates **experience**, **API/policy**, **scan workers**, **AI intelligence**, and **data/integrations**. This allows horizontal scale of scanners, safer isolation of network modules, and pluggable AI providers without rewriting the UI.

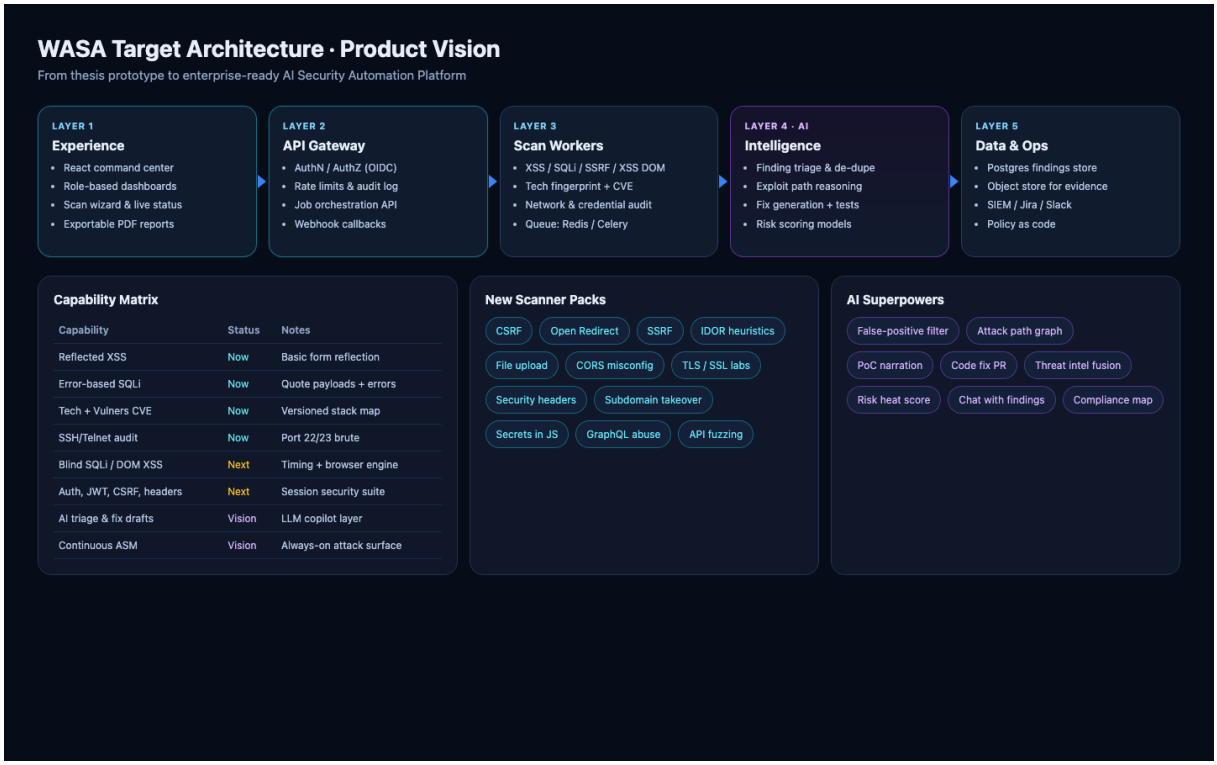

Figure 7 — Target layered architecture and capability matrix (Now / Next / Vision).

Recommended tech evolution

Area	Direction
Frontend	React 18+/Next.js, design system (tokens), charts, virtualized findings tables
API	FastAPI or hardened Flask, OpenAPI, JWT/OIDC, per-org API keys
Workers	Celery/RQ/Arq + Redis; browser pool (Playwright) for DOM XSS
Data	Postgres (findings), object store (evidence), optional OpenSearch
AI	Gateway with tool calling over findings DB; structured outputs for tickets
Ops	Docker Compose → K8s; secret manager; SBOM; CI SARIF gate

07 · Phased Roadmap

90-day to 12-month plan

Phase 0 — Stabilize & secure the thesis (2–3 weeks)

- Remove committed venv; add requirements.txt / lockfiles; env-based secrets; rotate exposed keys.
- Fix SQLi module imports/reliability; normalize all scanner outputs to a Finding schema (JSON).
- Add basic auth for API; disable network module unless LAB_MODE=true.
- Loading/error/empty states; disable buttons while scanning; cancel support.

Phase 1 — Product UX shell (1–2 months)

- Implement Command Center + Findings list + Finding detail from the mocks in this document.
- Scan history, project/target entities, PDF export of a single scan.
- Severity model, OWASP/CWE tags, evidence attachment for HTTP based modules.

Phase 2 — Scanner depth (2–4 months)

- Headless browser XSS; blind SQLi; security headers/TLS pack; secrets-in-JS.
- Authenticated scanning (cookie/session import).
- Async workers + progress events (SSE/WebSocket).

Phase 3 — AI copilot (parallel after schema exists)

- Summaries, prioritization, fix drafts, chat-with-findings RAG.
- Attack path graph v1 for multi-finding correlation.
- Human-in-the-loop controls and prompt/audit logging.

Phase 4 — Platform & GTM (6–12 months)

- SSO, multi-tenant, integrations, CI SARIF, continuous ASM light.
- Marketplace of scanner packs; on-prem option for regulated customers.
- Positioning: “AI AppSec for teams who outgrew one-off scripts.”

Phase	Focus	Exit criteria
0	Safety & schema	No secrets in git; Finding JSON; lab gate
1	UI shell	Dashboard + history + PDF users trust
2	Scanner depth	≥8 modules; async jobs; auth scan
3	AI layer	Triage + fixes reduce MTTR measurably
4	Platform	Multi-tenant pilot / CI integration

08 · Design System & UX Writing

Visual language

Use a dark operational canvas (`#0B1220`) with cyan/indigo accents for primary actions, rose/amber/blue/emerald for severity, Inter or system UI fonts, 12–16px radii, and card-based layout. Avoid playful illustrations in the scan path; reserve motion for progress and success confirmation only.

Figure 8 — Core color tokens for UI consistency.

Voice & microcopy

- Prefer precise verbs: *Run scan*, *Retest finding*, *Export evidence* — not “Hack now.”
- Network module copy must say **Credential security audit (lab only)**, never “Attack server.”
- Error: “We couldn’t reach the target (timeout). Check URL and network.” + Retry.
- Success: “Scan complete · 3 critical · 7 high · View prioritized fix plan.”

Component inventory to build first

- ScanBar, ModuleCard, SeverityChip, FindingRow, EvidenceBlock, AiPanel, SafetyConfirmModal, ProgressStream, EmptyState, ReportShell.

09 · Positioning, Metrics & Success

How WASA can win a niche

Enterprise scanners (Burp Enterprise, Invicti, etc.) are heavy and expensive. Raw scripts are free but unusable for non-experts. WASA can own the middle: **beautiful, explainable, AI-assisted automation** for labs, education, indie hackers, and small product teams — with a clear ethical boundary and export paths into professional workflows.

Success metrics

Metric	Target intent
Time-to-first-finding	< 60s on a demo DVWA-like target
False positive rate	Track & drive down with AI+rules; publish confidence
MTTR assist	% of criticals with accepted fix draft within 1 day
Activation	% users who complete second scan within 7 days
Safety	0 unauthenticated public abuse incidents; 100% lab-gated network runs
Trust	NPS / “would show this report to my manager” qualitative score

Immediate next actions (this week)

1. Rotate any committed API keys; scrub git history if needed.
2. Define Finding JSON schema shared by all scanners.
3. Prototype the Command Center UI in the existing React app (even static).
4. Wrap network module behind LAB_MODE + confirmation dialog.
5. Write one golden-path demo script (DVWA/Juice Shop) for UX testing.
6. Decide AI provider strategy (gateway + structured outputs) after schema lands.

Closing

WASA already proves the hardest early thesis claim: a person can drive meaningful security checks from a browser against a live target. The next leap is product craft — structured findings, safety, depth, and an AI layer that turns raw detections into decisions. With the UI direction and capability roadmap in this document, WASA can grow from a master's project into a distinctive security automation product that people trust enough to run every week.

> **Design thesis:** Security tools fail when they dump noise. WASA will win when every screen reduces cognitive load, every module declares its blast radius, and every finding ends in a fix.

Document artifacts: high-fidelity HTML/PNG mocks in docs/pdf-assets/ · regenerate via docs/generate_wasa_vision_pdf.py